MIS 215 · Week 1

Object-Oriented Programming in Java: The Complete Study Guide

Procedural vs. OOP · Classes & Objects · State & Behavior · Encapsulation · Nouns vs. Verbs · The Television Class · new · Stack & Heap · References

Based on: Prof. Dr. Ömür Akdemir's Week 1 lecture slides (Moving Beyond Procedural Code; The Core Real-World Analogies; The Nouns vs. Verbs Technique; Java Television Class Anatomy) and the Television/Student infographics.

How to use this guide: Read the sections in order once. Then, the night before the exam, read only the **Exam Trap** boxes (Section 11), do the practice questions (Section 12) without looking at the answers, and finish with the one-page cheat sheet (Section 13). Every code sample in this guide was compiled and run, so the outputs shown are exactly what Java prints.

#	Section	#	Section
1	Why OOP? The problem with procedural code	8	Memory: Stack vs. Heap, primitives vs. references
2	What is an object? State + Behavior	9	The Student example & constructors
3	Encapsulation (information hiding)	10	Message passing & inheritance
4	Class vs. Object (blueprint vs. house)	11	Top exam traps
5	The Rosetta Stone of analogies	12	Practice questions + answers
6	Nouns vs. Verbs: designing classes from text	13	One-page cheat sheet
7	The Television class, line by line + LivingRoom		

MEMORY HOOK

The whole week in one sentence: A **class** is a blueprint that describes what an object **knows** (fields = nouns = state) and what it **does** (methods = verbs = behavior); **new** builds a real object on the **heap**, and your variable is only a **remote control** (reference) pointing at it.

1. Why OOP? Moving Beyond Procedural Code

Slogan from the lecture: stop writing sequences of actions, start building systems of entities.

The essence of programming = controlling complexity

Brian Kernighan (creator of C) said that controlling complexity is the essence of computer programming. Human capacity for detail is limited, so we cannot solve a complex problem all at once: we must **decompose** it into smaller pieces. The question is *how* we decompose it. There are two answers:

The old way: Procedural decomposition

- **Stepwise refinement:** break a big task into a sequence of smaller actions. Example: *Draw Figure → Draw Triangle + Draw Rectangle*.
- **Verb-centric:** the program is a list of actions (verbs) that happen to passive, defenseless data: `doTask1() → doTask2() → doTask3()`.
- **Algorithms + Data Structures = Programs** (Niklaus Wirth, 1975): actions come first, data second.

The flaw in the procedural system (the "Global Data" picture)

Problem	Meaning	Real-life picture
Separation	Data and the actions that use it are kept totally apart.	Your money sits in an open box in the town square; anyone can walk up.
Vulnerability	Data is passive and unprotected; any function can change it.	process_order(), clear_cache(), adjust_inventory()... all touch the same box.
Fragility	Change the data's structure → hundreds of scattered functions break.	Rename one column, and 300 functions that used it stop working.

As systems grow, top-down logic becomes a tangled web of dependencies.

The new way: Object-Oriented Programming

OOP **reverses the priority**: **Data + Algorithms = Objects**. We bundle data with the actions that operate on it into one self-contained unit.

	Procedural	Object-Oriented
Focus	Verbs / Actions	Nouns / Entities
Approach	Top-down logic	Interacting components
Data	Passive & exposed	Active & protected
Analogy	Following a recipe step by step	Directing a kitchen staff

MEMORY HOOK

Recipe vs. Kitchen Staff. Procedural = *you* are the only cook: you personally chop, fry, plate, wash. OOP = you are the head chef: you say "*Grill, cook the steak*", "*Pastry, make dessert*". Each station manages its own tools and knows its own job. You give orders; you don't micromanage.

Wirth flipped: Algorithms + Data = Programs → Data + Algorithms = **Objects**.

EXAM TRAP

"OOP means the program is a sequential list of instructions." **FALSE.** In OOP the program is a **community of distinct, cooperating entities**, each managing its own state and behavior. By modeling the world through **nouns instead of verbs**, we conquer complexity.

2. What Is an Object? State + Behavior

KEY DEFINITION

An **object** is a self-sufficient programming entity that groups **related data** and **operations**. Every object has exactly two halves:

STATE = what it **KNOWS** → data → **fields / instance variables** → nouns/adjectives

BEHAVIOR = what it **DOES** → actions → **methods** → verbs

Object	State (knows)	Behavior (does)
Music player	currentVolume, color, speed	playMusic(), rotate()
Car	speed, fuelLevel	brake(), startEngine()
Television	brand, volumeLevel = 14, currentChannel = 30, isPowerOn = ON	turnOn(), volumeUp(), changeChannel()
Bank account	balance, owner	deposit(), checkCredit()
Circle ("Cookie")	name = "Cookie1", radius = 5.0	getArea() → 78.5

The State ⇄ Behavior feedback loop

State and behavior are not independent; they constantly affect each other:

- **Behavior modifies state:** pressing volumeUp() changes volume 14 → 15.
- **State dictates available behavior:** volumeUp() (or changeChannel()) cannot execute if power state is OFF.
- Methods often use the object's **own** state (Java keyword `this`).

The feedback loop in code

```
public void changeChannel(int newChannel) {
    if (isPowerOn) {                  // STATE decides if the behavior is allowed
        currentChannel = newChannel;  // BEHAVIOR changes the STATE
    } else {
        System.out.println("Error: TV is OFF!");
    }
}
```

MEMORY HOOK

"Knows vs. Does" test. Put the word in this sentence: "*The TV ___ .*" If it's "*The TV has a volume*" → field. If it's "*The TV can change channel*" → method. **Has = field, Can = method.**

Methods end with `()` because they are actions you *call*: `turnOn()`. Fields have no parentheses: `volumeLevel`.

3. Encapsulation (Information Hiding)

Encapsulation = the object hides its data inside and only lets the outside world interact through approved methods. In the slides it's drawn as a capsule: **the outer shell (verbs/methods) protects and interacts with the inner core (nouns/fields).**

Part	Java keyword	Rule
Private data (state)	`private String brand;`	Hidden inside. Cannot be directly manipulated by outside code.
Public methods (behavior)	`public void turnOn()`	The authorized interface. Other code must **ask permission** to change data through these methods.

Memorable example: why private matters

```
public class BankAccount {
    private double balance;              // hidden core

    public void deposit(double amount) { // public shell = the "ATM"
        if (amount > 0) {                // the object enforces its own rules
            balance = balance + amount;
        }
    }
}
// In another class:
// acc.balance = -1000000;   // COMPILE ERROR: balance has private access
// acc.deposit(500);         // OK: you asked the object politely
```

MEMORY HOOK

The ATM analogy. The bank's cash (balance) is behind a steel wall (`private`). You can't reach in and grab it; you use the ATM buttons (`public` methods), and the ATM checks the rules (PIN, enough money). That is encapsulation.

Remember: "Data private, methods public."

4. Class vs. Object: Blueprint vs. House

	The CLASS	The OBJECT (Instance)
Definition	The concept, template, or blueprint	The concrete entity, the instance
Memory	Holds **no** physical space; describes the structure	Takes up actual space in memory (the heap)
How many?	Only **one** exists	**Many** can exist, each with unique state
Real-world	Architectural blueprint, cookie cutter, recipe, blank form	Physical house, baked cookie, filled-in contact
Java	`public class Television { ... }`	`new Television()`

- **Blueprint:** you cannot live inside a blueprint. It gives the exact instructions to build a specific type of house, ensuring consistency before any materials are used.
- **Houses:** they share a structural foundation, but each has its own identity and data (address, paint color: red, brown, white).
- **Cookie cutter:** one template can stamp out infinite unique instances. Class (the cutter) → Action (`new`) → Objects (cookies).
- **Recipe card in a box:** describes state and behavior an object will have, but it is **NOT** an object.

KEY DEFINITION

Instantiation = the act of using a class (the cutter) to create a specific object (the cookie). In Java this is done with the keyword **`new`**. An object is also called an **instance** of a class.

Why classes? Uniformity at scale

Real software has thousands of objects. A class **enforces a uniform structure**: like the blank "New Contact" form on your phone, it guarantees every contact has the exact same fields (first name, last name, phone, email), even if you have 10,000 contacts. Each object holds **unique state** (Alice 555-0100, Bob 555-0200, Charlie 555-0300) while **sharing identical behaviors** (call(), edit()).

MEMORY HOOK

Same methods, different data. All Samsung and Sony TVs share the same buttons (methods) but each one is on a different channel (state). Behaviors are shared; state is personal.

EXAM TRAP

"A class takes up memory for its fields." → **No.** Fields get memory only when an **object** is created with new. One class, many objects, each with its **own copy** of the instance variables.

5. The Rosetta Stone of OOP

If you can fill in this table for any example, you understand the vocabulary. Memorize the four rows from the slides, and practice making your own:

The Class	The Object	The State (Instance Variables)	The Behavior (Methods)
Blueprint	House	Paint Color	Open Door()
Cookie Cutter	Cookie	Frosting Type	Take a Bite()
Circuit Diagram	Television	Current Channel	Change Channel()
Blank Form	Contact Entry	Phone Number	Call Contact()
Recipe	*Cookie*	*name, radius*	*getArea()*
Student class	*adem*	*name = "Adem", gpa = 3.8*	*printDetails()*

The three fundamentals: **(1) Classes are templates**: they define the structure but hold no physical space. **(2) Objects are instances**: concrete entities built from the template. **(3) Objects bundle state and behavior**: they combine what they know with what they can do.

6. Nouns vs. Verbs: Turning English into Classes

"When you receive a software requirement from a client, how do you decide what classes and methods to create? Look at the grammar!"

	NOUNS (İsimler)	VERBS (Fiiller)
Grammar	People, places, things, concepts, attributes	Actions, events, processes, commands
OOP equivalent	**Classes** (blueprints) or **Fields** (data/state)	**Methods** (functions/commands)
Purpose	Defines the state: what the system IS or KNOWS	Defines the behavior: what the system DOES
Examples	Student, Car, Balance, rowNumber	deposit(), printDetails(), startEngine()

The Noun Resolution Framework: Class or Field?

Not all nouns are equal. Run every noun through two questions:

- **Q1: Is it an independent entity with its own behaviors?** → YES → make it a **CLASS** (Customer, Car, Order).
- **Q2: Is it just a property, attribute, or measurement of another entity?** → YES → make it a **FIELD** inside that class (radius, name, shipping_address).

MEMORY HOOK

The "Can it DO something?" test. A *Card* can be dealt, added, removed → class. A card's *suit* can't do anything by itself; it only describes the card → field. **If a noun is just an adjective in disguise, it's a field.**

The "'s" test: if you can say "*the X's Y*", Y is usually a field of X: the Card's suit, the Seat's row number, the Student's GPA.

Case Study 1: E-Commerce

"*When a new **Item** is **added** to an **Order**, the Order object should be the one in charge. Later, Orders can be **shipped** or **canceled**. **Payments** are **applied** to Orders.*"

Nouns identify key entities; verbs describe their lifecycle. Result: **Order** { add(Item), ship(), cancel(), apply(Payment) }, **Item**, **Payment**.

Assigning the verbs: who owns add()?

When an Item is added to an Order, which class gets the method? **Rule: the class that has the primary responsibility for the data owns the method.** The Order knows how it stores and sorts its items, so Order owns the verb → **`order.addItem(item)`**, not item.addToOrder().

MEMORY HOOK

The "who holds the list?" rule. Whoever holds the collection owns add/remove. The Hand holds cards → Hand.addCard(). The Deck holds cards → Deck.shuffle(), Deck.dealCard(). The Order holds items → Order.addItem().

Case Study 2: The Card Game

*"A dealer will **shuffle** a Deck of Cards. The dealer will then **deal** a Card from the Deck. Cards can be **added** to or **removed** from a player's Hand. We also need to determine a Card's suit and value."*

Class	Fields	Methods	Why
Deck	(list of cards)	shuffle(), dealCard()	Deck holds the cards being shuffled/dealt
Hand	(list of cards)	addCard(), removeCard()	Hand holds the player's cards
Card	**suit, value**	-	suit and value are nouns but only **attributes** of a Card → fields, not classes!

Case Study 3: Cinema Booking System

*"A Cinema Booking System **stores** seat bookings for a Theater. A Customer **requests** a seat booking. The system **reserves** the Seat and the customer is given a row number and seat number."*

Class	Members
BookingSystem	storeBooking(), reserveSeat()
Customer	requestBooking()
Theater	(holds seats)
SeatBooking	fields: **rowNumber, seatNumber**

PROFESSOR'S EMPHASIS

OOP doesn't force you to invent an architecture from thin air. Read carefully: the structure is already hidden in the grammar. **Find the nouns, assign the verbs, and the blueprint builds itself.**

7. The Television Class, Line by Line

Television.java

```
// File: Television.java   (Blueprint class - NO main method!)
public class Television {

    // 1. STATE: Instance Variables (Fields)
    String  brand;           // e.g. "Sony"     -> default null
    int     currentChannel;  // e.g. 7          -> default 0
    int     volumeLevel;     // e.g. 10         -> default 0
    boolean isPowerOn;       // e.g. false      -> default false

    // 2. BEHAVIOR: Instance Methods
    public void turnOn() {
        isPowerOn = true;                       // updates state
        System.out.println(brand + " TV is now turned ON.");
    }

    public void changeChannel(int newChannel) {
        if (isPowerOn) {                        // checks state
            currentChannel = newChannel;        // manipulates state
            System.out.println(brand + " TV channel changed to: " + currentChannel);
        } else {
            System.out.println("Error: Cannot change channel because " + brand + " TV is OFF!");
        }
    }

    public void displayStatus() {
        System.out.println("[" + brand + " TV] Power: " + isPowerOn +
                           " | Channel: " + currentChannel +
                           " | Volume: " + volumeLevel);
    }
}
```

Piece of code	Component type	Real-world concept
`public class Television`	Class declaration	Master schematic for all TVs
`int volumeLevel;`	Instance variable (field)	A physical setting or attribute
`public void turnOn()`	Instance method	A button on the remote
`int newChannel` (in parentheses)	Parameter	The number you type on the remote
`void`	Return type	Method gives nothing back; it just acts

Anatomy of a method header

```
  public     void        changeChannel   ( int newChannel )
  ------     ----        -------------     --------------
  access     return      method name       parameter (input)
  modifier   type        (a verb!)         type + name
```

The "No Main Method" Rule: the Blueprint Constraint

A blueprint class **describes a design; it does not run itself**. Television.java has **no** main method. Execution happens in a separate **client class** (driver class) that **requires** `public static void main(String[] args)`. That client creates objects with new and calls their methods.

What really happens if you try to RUN the blueprint (tested)

```
$ java Television
Error: Main method not found in class Television, please define the main method as:
   public static void main(String[] args)
```

MEMORY HOOK

A blueprint can't cook dinner. Television.java is the factory drawing; LivingRoom.java is the person in the living room who buys TVs and presses buttons. **Template = NO main. Execution = REQUIRES main.**

Note: Television.java still **compiles** fine. It only fails when you try to **run** it.

The client class: LivingRoom.java

LivingRoom.java (client class: HAS main)

```
public class LivingRoom {
    public static void main(String[] args) {
        // 1. Instantiate first object
        Television livingRoomTv = new Television();
        livingRoomTv.brand = "Samsung";
        livingRoomTv.volumeLevel = 15;

        // 2. Instantiate second object
        Television bedroomTv = new Television();
        bedroomTv.brand = "Sony";
        bedroomTv.volumeLevel = 8;

        // 3. Interact with object 1
        livingRoomTv.turnOn();
        livingRoomTv.changeChannel(7);
        livingRoomTv.displayStatus();
        System.out.println("----------------------------------------");

        // 4. Interact with object 2 (independent state!)
        bedroomTv.changeChannel(10);   // error: power is OFF
        bedroomTv.displayStatus();
    }
}
```

Console output (verified)

```
Samsung TV is now turned ON.
Samsung TV channel changed to: 7
[Samsung TV] Power: true | Channel: 7 | Volume: 15
----------------------------------------
Error: Cannot change channel because Sony TV is OFF!
[Sony TV] Power: false | Channel: 0 | Volume: 8
```

Trace it like an exam question

Line	livingRoomTv (Samsung)	bedroomTv (Sony)	Why
after both new + setup	power false, ch 0, vol 15	power false, ch 0, vol 8	defaults: false, 0
`livingRoomTv.turnOn()`	**power true**	unchanged	only Samsung's state changes
`livingRoomTv.changeChannel(7)`	**ch 7**	unchanged	power is on → allowed
`bedroomTv.changeChannel(10)`	unchanged	**still ch 0**, error printed	Sony never turned on

EXAM TRAP

Why is Sony's channel 0, not 10? Because changeChannel only changes state if isPowerOn is true. Sony's isPowerOn is still the default **false**. Turning on the Samsung did **not** turn on the Sony: each object has its own independent state.

Why 0? No one ever set it; Java gives int fields the default value 0.

Dot notation

`objectReference.member` = "go to the object this remote points to, and use this member". `livingRoomTv.brand = "Samsung";` sets a field; `livingRoomTv.turnOn();` calls (sends a message to) a method.

8. Memory: What new Really Does (Stack, Heap, References)

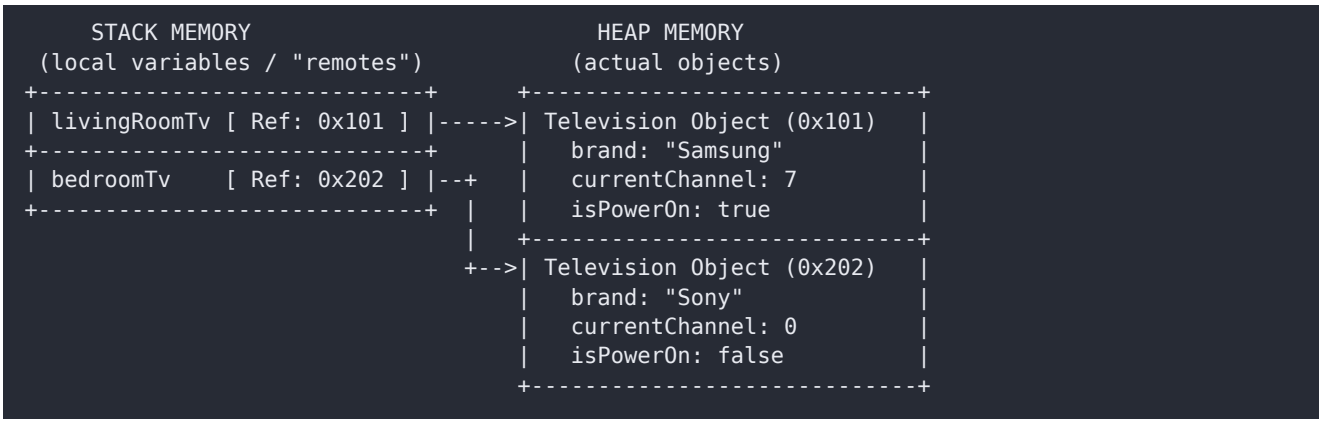
The three steps of instantiation ("the factory")

- **Step 1:** new triggers the **constructor**, assembling the object in memory.
- **Step 2:** the object lives in a memory area called the **Garbage-Collectible Heap**.
- **Step 3:** when the object is no longer needed (nothing points to it), the **JVM automatically** throws it out to reclaim space (**garbage collection**).

Reading one line of instantiation

```
Television livingRoomTv  =  new Television();
----------  ------------  -  ----------------
 1. declare  reference      3. build object on the HEAP
    type     variable on       (constructor runs)
             the STACK
                       2. "=" stores the object's ADDRESS in the variable
```

Stack vs. Heap (the professor's board diagram)



	STACK	HEAP
Stores	Local / reference variables (livingRoomTv, bedroomTv)	The actual objects created with new
Analogy	Remote controls in your hand	The big TVs on the wall
Cleaned up by	Automatically when the method ends	Garbage collector (JVM) when unreachable

Primitives vs. References

	Primitive types	Reference types
Examples	int, double, boolean (also char, long...)	String, Television, Student: any class
Variable holds	The **actual value** (the box contains 5)	An **address** pointing to the object on the heap
Default as a field	int → 0, double → 0.0, boolean → false	**null** (points to nothing)
Analogy	A box with the thing inside	A remote control for a massive TV

MEMORY HOOK

THE REMOTE CONTROL ANALOGY (most important idea of the week): Variables never hold objects. Objects are massive and complex, so the variable only holds a **reference**: a remote control pointing to the TV on the heap. `livingRoomTv.turnOn()` = press the ON button on the remote; the signal goes to the TV, and the TV turns itself on.

Consequence #1: copying a reference copies the remote, not the TV (aliasing)

Alias.java

```
Television a = new Television();
a.brand = "LG";
Television b = a;            // second remote, SAME TV
b.turnOn();                  // press ON with remote b...
a.displayStatus();           // ...TV seen through remote a is ON too
System.out.println(a == b);  // same object?

Television c = new Television();
c.brand = "LG";
System.out.println(a == c);  // identical data, but a DIFFERENT TV

int x = 5;  int y = x;  y = 99;
System.out.println(x);       // primitives copy the VALUE
```

Output (verified)

```
LG TV is now turned ON.
[LG TV] Power: true | Channel: 0 | Volume: 0
true
false
5
```

EXAM TRAP

== on objects compares addresses (remotes), not contents. Two TVs with the same brand are still two different objects → false. Only two remotes to the *same* TV give true.

Copying a primitive copies the value (x stays 5). **Copying a reference** shares the object.

Consequence #2: null and default values

Defaults and null (verified)

```
Television d = new Television();
d.displayStatus();            // [null TV] Power: false | Channel: 0 | Volume: 0

Television t = null;          // a remote paired with NO TV
t.turnOn();                   // NullPointerException at runtime!
```

MEMORY HOOK

null = a remote with no TV. Press any button → nothing to send the signal to → `NullPointerException`.

Garbage collection: if you throw away the last remote (`a = null;`) the TV is unreachable; the JVM's "janitor" eventually hauls it away. You never delete objects manually in Java.

9. The Student Example & Constructors

Student.java + use (verified)

```
public class Student {
    String name;   // the student's name   (field: String)
    double gpa;    // grade point average  (field: double)

    // Constructor: same name as class, NO return type
    public Student(String name, double gpa) {
        this.name = name;   // this.name = the field; name = the parameter
        this.gpa  = gpa;
    }

    void printDetails() {   // behavior
        System.out.println("Name: " + name + ", GPA: " + gpa);
    }
}

// In a client class with main:
Student adem = new Student("Adem", 3.8);
adem.printDetails();        // Name: Adem, GPA: 3.8
```

What is a constructor?

- A special block that runs **automatically when** new **is used**, to set up (initialize) the new object's state.
- Its name is **exactly the class name**, and it has **no return type** (not even void).
- If you write **no** constructor, Java gives a free **default constructor** with no parameters. That's why `new Television()` works even though Television has no constructor written.
- `new Student("Adem", 3.8)` only compiles if the class **defines** a constructor taking (String, double). The infographic shows only the fields with "// ..."; the constructor is hidden in that part.

> **MEMORY HOOK**
> **Constructor = the birth certificate.** new = the baby is born; the constructor fills in the name and starting values at birth. Without arguments (default constructor), the baby is born with blank papers (null, 0, false), and you fill them in later with dot notation, like livingRoomTv.brand = "Samsung".

Variable vs. object ("Değişken ve Referans İlişkisi")

The variable `adem` does **not** contain the object itself; it holds the **reference** to the object in memory. The Student object (name "Adem", gpa 3.8) lives on the heap.

Syntax rules from the slides

- All code must live inside the class's braces: `public class Student { ... }`.
- Every statement ends with a semicolon `;`.
- File name must match the public class: `Student.java` contains `public class Student`.
- Comments: `// single line`. Java is **case-sensitive**: Student ≠ student.

> **PROFESSOR'S EMPHASIS**
> The "Exit Ticket" infographic shows two ways to state the idea: `new Student();` "allocate a new Student object in memory", and `Student adem = new Student("Adem", 3.8);` creating the specific object 'adem'.

10. Message Passing & Inheritance

Message passing: objects talking to objects

Instead of a central master function micromanaging data, client code **sends messages to objects**. Example: `account.checkCredit()`: you don't calculate the credit yourself; you **ask** the Account object to do it based on its own hidden rules. Every method call `obj.method()` is a message.

> **MEMORY HOOK**
> **Don't do it, ask it.** You don't open the TV and rewire it to change the channel; you send the message changeChannel(7) and let the TV handle the details.

Inheritance: the power of extension (preview)

- **Code reuse:** if a class almost fits your needs, don't copy-paste it: **extend** it.
- A **subclass** inherits all state and behavior from a **superclass**, eliminating redundant code. Example: Circle, Square, Triangle extend **Shape**.
- **Fun fact:** in Java, every class implicitly extends the ultimate superclass called **`Object`**.

extends keyword

```
public class Shape  { String color; }
public class Circle extends Shape { double radius; }  // Circle HAS color too
```

MEMORY HOOK

"is-a" test: A Circle *is a* Shape → inheritance fits. A Car *has an* Engine → that's a field, not inheritance.

11. Top Exam Traps (Read These Twice)

Wrong belief	The truth
✗ A class is an object.	✓ No. Class = blueprint/recipe (one). Object = instance built from it with new (many).
✗ A class takes memory for its data.	✓ No. Classes hold no physical space; objects take up actual memory on the heap.
✗ The blueprint class needs a main method.	✓ No. Television.java has NO main. The client class (LivingRoom) requires main.
✗ A variable stores the object.	✓ No. A reference variable stores the object's address (a remote control). Only primitives store actual values.
✗ Primitive types include String.	✓ No. Primitives are int, double, boolean (char, long...). String is a class → reference type.
✗ Every noun becomes a class.	✓ No. Nouns that are attributes (suit, value, rowNumber, radius) become fields.
✗ Item owns add() when added to an Order.	✓ No. The class responsible for the data owns the method → Order.addItem(item).
✗ Changing one TV's state affects other TVs.	✓ No. Each object has its own copy of instance variables (unless two variables reference the same object).
✗ Default value of an uninitialized int / boolean / String field.	✓ 0 / false / null. (Sony: Channel 0, Power false.)
✗ Procedural = nouns, OOP = verbs.	✓ Reversed! Procedural is verb-centric; OOP is noun/entity-centric.
✗ Who said what?	✓ Kernighan: "controlling complexity is the essence of programming". Wirth (1975): "Algorithms + Data Structures = Programs".
✗ Who deletes objects?	✓ The JVM's garbage collector, automatically, when an object is unreachable.
✗ Constructor has return type void.	✓ No. A constructor has NO return type and the same name as the class.
✗ a == b is true if two objects have the same data.	✓ No. == on objects checks if both references point to the same object.
✗ Encapsulation means making everything public.	✓ Opposite: data private (hidden), public methods as the authorized interface.

12. Practice Questions

Cover the answers (next page) and try each one first.

A. Multiple choice

1. Which describes the Object-Oriented approach? (a) Top-down logic (b) Verbs/actions (c) Interacting components with active, protected data (d) Passive, exposed data

2. In `private int volumeLevel;`, volumeLevel is a(n): (a) method (b) instance variable (c) class (d) constructor

3. Where does the object created by `new Television()` live? (a) Stack (b) Heap (c) Inside the class file (d) Inside the variable

4. "A Card's suit and value": suit should be (a) a class (b) a method (c) a field of Card (d) a main method

5. Which is NOT a primitive type? (a) int (b) boolean (c) double (d) String

6. Rosetta Stone: Cookie Cutter → Cookie → Frosting Type → ? (a) Take a Bite() (b) Open Door() (c) Recipe (d) Oven

7. "The essence of programming is controlling complexity" was said by (a) Wirth (b) Gosling (c) Kernighan (d) Turing

B. True / False

8. A blueprint class like Television.java must contain a main method.

9. Two Television objects created with new share the same isPowerOn variable.

10. In Java, every class implicitly extends Object.

11. State dictates available behavior.

C. Code tracing: what is printed?

Question 12

```
Television tv1 = new Television();
tv1.brand = "Vestel";
Television tv2 = tv1;
tv2.volumeLevel = 20;
tv1.changeChannel(5);
tv2.turnOn();
tv1.changeChannel(9);
tv1.displayStatus();
System.out.println(tv1 == tv2);
```

D. Design (nouns vs. verbs)

13. "*A Library lends Books to Members. A Member can borrow and return a Book. Each Book has a title, an author and an ISBN. The Library can search for a Book by title.*" Identify classes, fields and methods, and say who owns borrow().

14. Explain in two sentences why `java Television` fails but `java LivingRoom` works.

Answers

Q	Answer & reason
1	**(c)**: OOP = interacting components; data is active & protected.
2	**(b)**: a noun describing what the TV knows → instance variable (field).
3	**(b)** Heap. The variable on the stack only holds the reference.
4	**(c)**: suit is an attribute of a Card, not an independent entity.
5	**(d)** String is a class → reference type.
6	**(a)** Take a Bite(): the behavior column.
7	**(c)** Brian Kernighan, creator of C.
8	**False**: blueprint = no main; the client class has main.
9	**False**: each object has its own copy of every instance variable.
10	**True**: the cosmic superclass Object.
11	**True**: e.g. changeChannel() is blocked when isPowerOn is false.

12. Output (tv1 and tv2 are two remotes for ONE Vestel TV):

```
Error: Cannot change channel because Vestel TV is OFF!
Vestel TV is now turned ON.
Vestel TV channel changed to: 9
[Vestel TV] Power: true | Channel: 9 | Volume: 20
true
```

Why: changeChannel(5) fails because power is still false. tv2.turnOn() turns on the *same* TV, so changeChannel(9) works. Volume 20 was set through tv2 but is visible through tv1. == is true because both hold the same address.

13. Classes: **Library** { search(title) }, **Member** { borrow(book), returnBook(book) }, **Book** { fields: title, author, isbn }. title/author/ISBN are attributes → fields. borrow() belongs to the class responsible for that data: Member (keeps its borrowed books) is a good answer; you could also argue Library.lend(book, member) if the Library keeps the loan records. State the reasoning!

14. Television is only a blueprint: it describes TVs but has no main method, so the JVM has no starting point ("Main method not found"). LivingRoom has `public static void main(String[] args)`, so it can run, create Television objects with new, and call their methods.

13. One-Page Cheat Sheet

Concept	Remember it as	Java
Procedural	Recipe; verbs act on passive global data; fragile	doTask1() → doTask2()
OOP	Kitchen staff; nouns; Data + Algorithms = Objects	objects send messages
Class	Blueprint / cookie cutter / recipe / blank form (only one, no memory)	public class Television { }
Object / instance	House / cookie / filled contact (many, real memory)	new Television()
Instantiation	Stamping a cookie with the cutter	new
State	What it KNOWS; HAS-a → field (noun)	int volumeLevel;
Behavior	What it DOES; CAN → method (verb)	public void turnOn()
Feedback loop	Behavior modifies state; state dictates behavior	if (isPowerOn) {...}
Encapsulation	ATM: data private, methods public	private / public
Noun → ?	Independent entity with behavior → class; attribute → field	Card / suit
Verb owner	Class responsible for the data owns the method	order.addItem(item)
Blueprint rule	Template: NO main. Client/driver: REQUIRES main	public static void main(String[] args)
Reference	A remote control holding an address	Television tv = ...
Primitive	A box holding the actual value	int, double, boolean
Stack	Remotes (local reference variables)	livingRoomTv [0x101]
Heap	The real TVs (objects); garbage-collectible	Television@0x101
Garbage collection	JVM janitor removes unreachable objects	automatic
null	Remote with no TV → NullPointerException	Television t = null;
Defaults	int 0 · double 0.0 · boolean false · objects null	Sony: Channel 0
Aliasing	Copy the remote, not the TV	b = a; a == b → true
Constructor	Birth certificate; class name, no return type	public Student(String n, double g)
Message passing	Don't do it, ask it	account.checkCredit()
Inheritance	is-a; extend instead of copy-paste; all extend Object	class Circle extends Shape

MEMORY HOOK

Final 5-second recap: Nouns → classes & fields (KNOWS). **Verbs** → methods (DOES). **Class** = blueprint (no main). **new** = build on the heap. **Variable** = remote. Good luck!